GPS-Loss Path Retrace: Project Report
Which Sensors Bring a Drone Home?

MTech Capstone Project, GPS-Denied Drone Navigation, IIT Madras

September 2026, branch *optimal-retracement*

1. Objective

Find out which onboard sensors a quadcopter needs to get home after GPS fails. Two return strategies were tested. Exp1 (dead reckoning): integrate gyro and accelerometer from the last known fix and fly the home arrow. Exp2 (breadcrumb reversal): re-fly the logged 2 Hz trail backwards, steering toward each point in turn. Six sensor sets ran through both, from IMU-only (C1) up to IMU plus compass, barometer, thrust, battery and zero-velocity updates (C6).

Set	Contains	Expected benefit
C1	gyro + accel	baseline; minimum for inertial navigation
C2	+ compass yaw	stops yaw random walk
C3	+ barometer	holds the vertical channel
C4	+ thrust drag model	caps velocity error
C5	+ battery current	thrust scaling at saturation (no effect found)
C6	+ ZUPT	kills drift while hovering

2. Setup

Same estimator core in all three testbeds, so result differences come from data, not code: a quaternion strapdown integrator with static-window calibration (biases, hover throttle, one compass offset), compass yaw blending, baro blending, rotor-drag damping and ZUPT. Testbed 1: two ArduPilot logs (Flight A 48 MB, Flight B 79 MB) replayed under simulated 10/30/60 s outages against gated GPS truth (HDop under 1.0, 10 or more sats), 72 runs total. Testbed 2: a 120 s scripted profile (hover, 8 m/s legs, banked turns, climb; 801 m) with machine-exact 100 Hz sensors, deleting one factor at a time (F1-F6). Testbed 3: a 20 m square at 5 m flown live in PX4 SITL (Gazebo taif_test4 plus MAVROS, 114.5 m, PX4 home miss 0.36 m), replayed per combo through a ROS estimator node, plus a perfect-sensor rerun of the same square. Scripts: run_matrix.py, optimal_world.py, sitl_optimal.py, commander.py, retrace_node.py, replay_all.sh, RUN_SITL_LOOP.ps1.

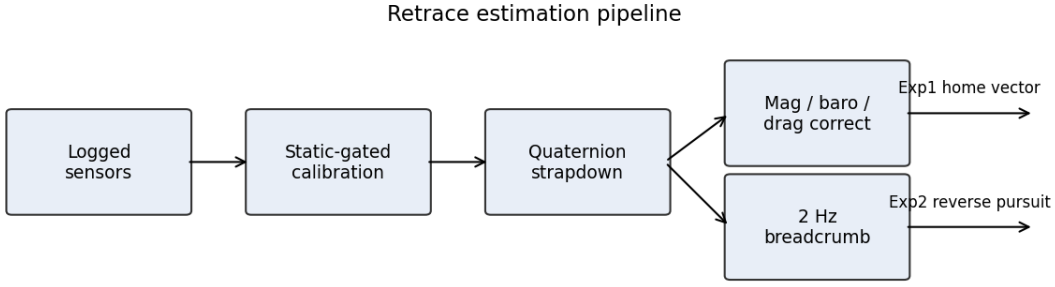

Figure 1. Signal flow: calibration, propagation, per-set aids, Exp1 vector and Exp2 pursuit.

3. Results

3.1 Perfect world (801 m, exact sensors)

Run	Removed	ATE (m)	Yaw err (rad)	Outcome
F1 full	nothing	0.21	0.00004	baseline
F4 / F6	compass / baro	0.22	0.00000	no loss
F2 / F3	gyro / accel	232 / frozen	0.075 / exact	each core sensor necessary
F5	thrust proxy	365	exact	velocity aid matters even here

Takeaway: with exact sensors the core pair does everything. Each is individually necessary.

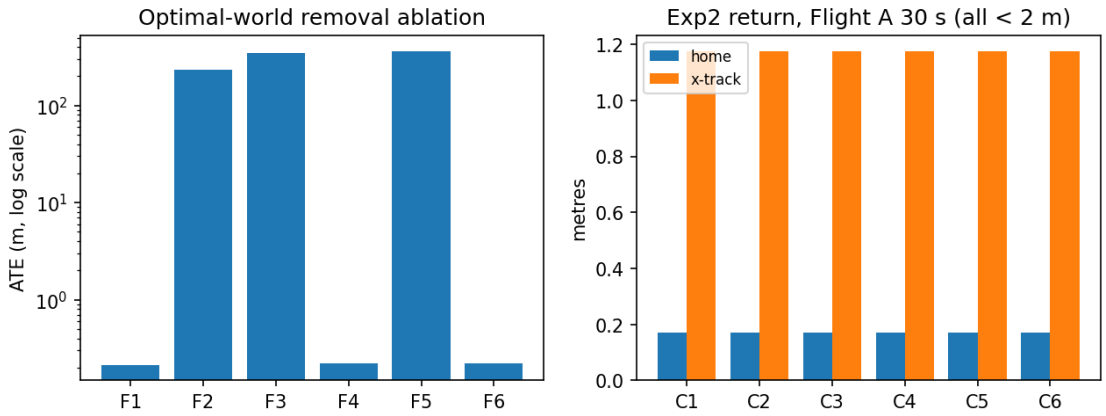

Figure 2. Removal ablation, log scale (left); Exp2 return on Flight A 30 s leg (right).

3.2 Real logs (ATE in metres)

Log/set	10 s	30 s	60 s
A C1 (IMU)	11.9	120.7	460.8
A C3 (+mag+baro)	12.9	119.0	433.4
A C4 (+drag)	4.3	8.8	14.8
A C6 (+ZUPT)	3.9	4.9	7.3
B C1 (IMU)	2.4	30.3	223.2
B C4 (+drag)	1.7	11.4	43.1
B C6 (+ZUPT)	0.3	0.6	7.4

Takeaways: drag is the biggest win (13x on A at 30 s) because capped velocity error grows position linearly instead of quadratically. Battery adds nothing. ZUPT needs real hover (19x on B, nil on A). Compass halves yaw error on A (0.47 to 0.23 rad) but an uncalibrated 0.3 rad bias hurts B (0.00 to 0.30 rad): calibrate or drop it. Breadcrumb return lands under 2 m for every set on both flights (optimal: 0.02 m) since pursuit re-aims each step.

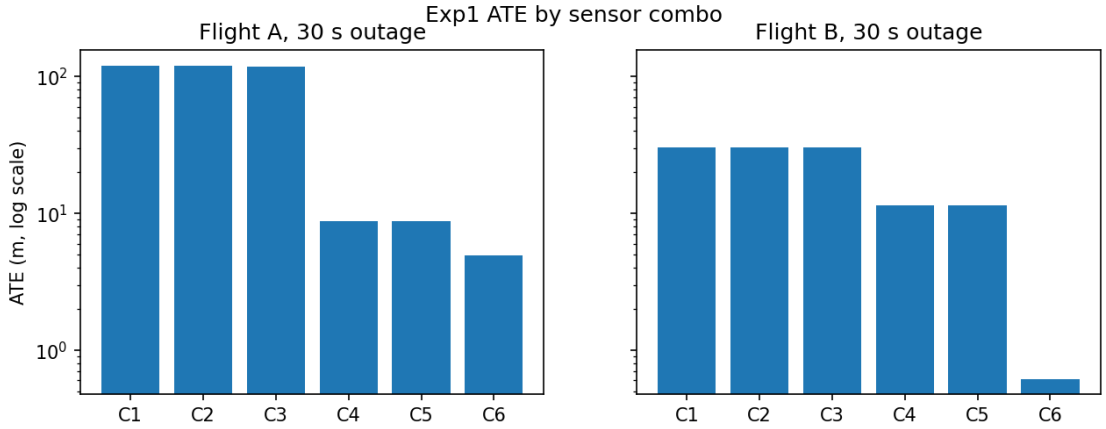

Figure 3. Exp1 ATE by set, 30 s outage, log scale.

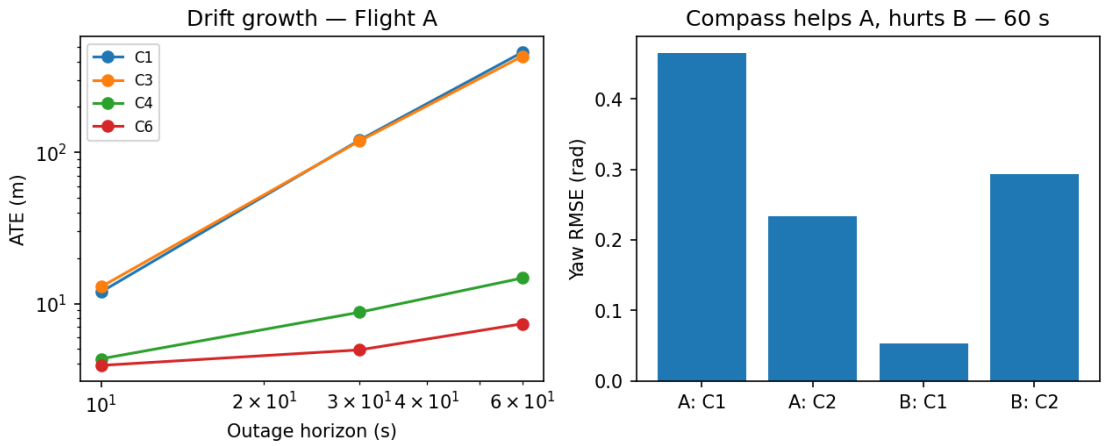

Figure 4. Drift growth with outage length (left); yaw error at 60 s (right).

3.3 Live SITL square, raw sensors (114 m)

Set	ATE 3D (m)	Horiz (m)	Vert (m)
C1 (IMU)	2690	2690	23.9
C2 (+mag)	2677	2677	24.6
C3 (+baro)	2665	2665	0.3
C4 (+drag)	423	423	0.3

Takeaway: identical ranking as real logs (compass nil, baro vertical-only 24 to 0.3 m, drag 6.3x), roughly 50x scale from measured sensor dirt: +0.46 m/s2 cruise rectification bias, spikes to 1137 m/s2 and 37 rad/s at 49 Hz, EKF height wobble. Quote the ranking, not the magnitudes. C5/C6 omitted (nil offline, no hover in square).

3.4 Same square, perfect sensors (80 m profile)

Set	ATE (m)	Home (m)
C1 (IMU)	0.006	0.00
C3 (+baro)	0.005	0.00
C4 (+drag)	4.97	0.21
C6 (+ZUPT)	4.99	0.00

Takeaway: 6 mm out of the same estimator on the same geometry. Raw-vs-perfect is 2690 m vs 0.006 m, so the gap is sensor dirt, measured. Drag costs 5 m here only because stop-and-go corners keep it transient against drag-free truth.

4. Issues faced and fixes

Issue	Symptom	Fix
ATT in degrees read as radians	errors 10-20x too big	convert on load (57x factor)
World-frame delta as body rate	yaw RMSE 1.54 rad	body-frame update (0.001 rad)
Yaw RMSE unwrapped	about 35 rad reported	circular RMSE
Tautological Exp2 scorer	perfect scores always	rebuilt as pursuit
Arming checks at -1	PX4 never arms	lenient 10.0 plus save
MAVROS transmit to dead port	services unusable	fcu_url to onboard listener 14580
Pymavlink direct TX dead	takeoff never happens	MAVROS-native commander
Instant 0-5 m takeoff step	10 s IMU storm	ramped 0.5 m climb
FLU/ENU vs FRD/NED mix	estimate 1097 m off	boundary conversion plus full-attitude init
Level-rest bias vs 4-6 deg tilt	about 1 m/s2 leak	rotate gravity by init attitude
Sim-time replay pitfalls	teleporting tracks, skipped cal	wall clock, -r 1, MAVROS dead, one publisher, topic-filtered play

SITL also degrades on the ground: EKF GPS drift re-fails preflight after about ten minutes, so reboot PX4 and fly fast. MAVROS IMU at 49 Hz carries the vibration of the sim motors; the

estimator skips samples over 25 m/s2 or 3 rad/s with time carried forward.

5. Conclusion and next steps

Log gyro and accel fast; they are necessary and sufficient when clean. Log motor commands; the drag model on them is the biggest real-world win measured here (13x). Calibrate the compass or drop it. Keep baro and a 2 Hz breadcrumb trail. Analytic truth, two real logs and a live square all agree.

Next: per-airframe drag-gain tuning, velocity aiding (optical flow/VIO) since blind dead reckoning (8-43 m at 30-60 s) is still too poor to fly home on, and mag re-test in SITL. Reproduce everything from the repo root: run_matrix.py, optimal_world.py, sitl_optimal.py, then sitl/README.md Part 2 and RUN_SITL_LOOP.ps1 for live runs.

References

[1] ArduPilot, Dead Reckoning Failsafe, Copter documentation, ardupilot.org.

[2] D. Titterton and J. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed., IET, 2004.

[3] P. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed., Artech House, 2013.

[4] RIOTU Lab, gps_denied_navigation_sim: PX4 SITL and Gazebo GPS-denied benchmark, github.com/riotu-lab.